

regex with Unicode character types

Table of Contents

I.

[Introduction and Motivation](#)

II.

[Scope and Impact on the Standard](#)

III.

[<regex> with char16_t](#)

IV.

[Technical Specifications](#)

V.

[Relevant Issues](#)

VI.

[References](#)

I. Introduction and Motivation

Among the four character types that C++ has, only `char` and `wchar_t` can be used with the regular expression library in the C++ standard. Because of this, operations involving regular expression matching and searching against a Unicode string are available only in such environments as the value of `char` or `wchar_t` denotes a UTF-32 character.

It is unfortunate and inconvenient that while C++ has two character types, two string classes dedicated to Unicode (`char16_t` and `char32_t`, `u16string` and `u32string`), and a regular expression library (`regex`), they cannot be used together in all implementations.

In this paper it is proposed that the regular expression library in the C++ standard (henceforth, `<regex>`) should support sequences of Unicode character types at least as the same level as sequences of `char` and `wchar_t`.

II. Scope and Impact on the Standard

Since there are different problems in using `<regex>` with `char16_t` or `char32_t`, different measures are required for each of them:

<code><regex></code> with <code>char32_t</code>
The value of <code>char32_t</code> is practically a Unicode code point itself. It should be adaptable to <code><regex></code> in essence without special treatment, however, <code>basic_regex<char32_t></code> is unavailable in most implementations based on the current standard. Its core reason is that although inside the class it tries to use <code>regex_traits<char32_t></code>, this is not available because it depends on several classes in <code><locale></code>, namely <code>ctype<char32_t></code>, <code>collate<char32_t></code>, and <code>collate_byname<char32_t></code> for which specializations are not defined in the standard. Thus, for <code><regex></code> to support <code>char32_t</code>, it is proposed to define specializations of these classes for <code>char32_t</code> in the standard.
<code><regex></code> with <code>char16_t</code>
Use of <code><regex></code> with <code>char16_t</code> has the following problems: - Regular expressions that represent a set of characters, such as <code>[\u0000-\uFFFF]</code> (character class), <code>.</code> (dot atom), <code>\S</code> (predefined character class) etc. can match a half of a surrogate pair instead of the whole pair that represents one Unicode character, since comparison is performed conceptually between a code unit in the sequence of regular expressions and a code unit in the input sequence passed to an algorithm. - Like the case of <code>char32_t</code>, the specializations <code>regex_traits<char16_t></code>, <code>ctype<char16_t></code>, <code>collate<char16_t></code>, and <code>collate_byname<char16_t></code> are not available. However, unlike <code>char32_t</code>, it is difficult to define appropriately

specializations of `cctype` for `char16_t` because it has some member functions that take an argument of `charT`, i.e., `char16_t` and return a value of the same type. This means that such functions cannot deal with a surrogate pair, and icase matching depending on one of such functions, `tolower()`, is not performed correctly by the algorithms of `<regex>`.

Note: UCS-2 is already [obsolete in the Unicode standard](#) and deprecated in ISO/IEC 10646. Newly added features must not support UCS-2 explicitly.

For `<regex>` to support `char16_t`, therefore, special treatments would be required. This is discussed in the next section, but in any case the existing libraries except `<regex>` would not be affected at all.

There might be demand for more full-featured Unicode regular expression support like the ones described in [UTS #18](#) to get into the C++ standard. But I propose, as a first step, for `<regex>` to support sequences of Unicode character types as the same level as sequences of `char` and `wchar_t`, based upon the following reasons:

- It can easily be imagined that regular expression matching operations considering normalization, composite characters, variation sequences, grapheme clusters, etc. are very slow. Even if they are supported in the future, it would be indispensable *as one option* for `<regex>` to support simple character-by-character comparison for `char16_t` and `char32_t`, as well as for `char` and `wchar_t`.
- These (normalization etc.) are not features specific to regular expressions but used generally in text matching, comparison and searching. They need to be considered in a comprehensive Unicode proposal.

Note: As of October 2015, among six regular expression grammars referred to by the C++ standard, only RegExp of ECMAScript has explicit Unicode support and it performs character-by-character comparison where each character is either a code point or a code unit of UTF-16, depending upon whether the `/u` flag is set or not.

III. `<regex>` with `char16_t`

There are two options for `char16_t` support:

1. Provide UTF-16 to UTF-32 converting iterator

In this option the C++ standard does not support `std::u16regex`, but defines a bidirectional iterator that converts UTF-16 to UTF-32 on the fly for the algorithms of `<regex>`. This takes pointers or iterators pointing to the sequence [begin, end) of UTF-16 as input, its `operator*()` returns a value of `char32_t`, and its `operator++()` and `operator--()` move its position to the next and previous character respectively in the sequence. A very rough sketch of it is illustrated as follows:

```
template<class BidiIterator>
struct regex_u16u32conv_iterator
{
public:
    typedef bidirectional_iterator_tag iterator_category;

    regex_u16u32conv_iterator(BidiIterator begin, BidiIterator end) : boi(begin), eoi(end)
    {
    }

    char32_t operator*()
    {
        if ((*boi & 0xdc00) == 0xd800)
        {
            BidiIterator trail = boi;
            if (++trail != eoi)
                return static_cast<char32_t>((( *boi & 0x3ff) << 10 | (*trail & 0x3ff)) + 0x10000);
        }
        return static_cast<char32_t>(*boi);
    }

    regex_u16u32conv_iterator &operator++()
    {
        ++boi;
        if (boi != eoi && (*boi & 0xdc00) == 0xdc00)
            ++boi;

        return *this;
    }
};
```

```

    }

    bool operator==(const regex_u16u32conv_iterator &right) const
    {
        return boi == right.boi && eoi == right.eoi;
    }

    operator BidiIterator() const
    {
        return boi;
    }

    // other members...

private:
    BidiIterator boi;
    BidiIterator eoi;
};
typedef regex_u16u32conv_iterator<char16_t*> regex_u16cu32conv_iterator;
typedef regex_u16u32conv_iterator<u16string::iterator> regex_u16su32conv_iterator;

char16_t u16chars[] = u"\u3000\u000010000\u0040"; // 0x3000, 0xd800, 0xdc00, 0x0040
regex_u16cu32conv_iterator u16tou32(u16chars, u16chars + 4);
*u16tou32; // returns 0x3000 of char32_t
++u16tou32;
*u16tou32; // returns 0x10000 of char32_t
++u16tou32;
*u16tou32; // returns 0x40 of char32_t

// A sequence of regular expressions in UTF-16 needs to be converted
// into UTF-32 prior to passed to u32regex.
u32string u32restr = U"(abc|def)[ghi]";
u32regex u32re(u32restr);

u16string u16text = u" long long text encoded in UTF-16... ";
regex_u16su32conv_iterator bos(u16text.begin(), u16text.end());
regex_u16su32conv_iterator eos(u16text.end(), u16text.end());
regex_search(bos, eos, u32re);

```

This does not need to satisfy strictly all the requirements of the bidirectional iterator, but only needs to be recognized so by all the algorithms of <regex>.

An advantage of this approach is that a similar iterator can be provided for UTF-8 to UTF-32 conversion, too. It is possible to support all UTFs (UTF-32, UTF-16, and UTF-8) by the combination of adding support for `char32_t` to <regex> and defining converting iterators.

A disadvantage is that matching operations are likely to be slow, since all code units are translated into UTF-32 through this iterator every time they are accessed in regular expression algorithms. Clearly, it would be faster than the way of this option to convert the input sequence of UTF-16 into UTF-32 in advance of passing it to `u32regex` or algorithms, if it is possible.

2. Do nothing for <regex> with `char16_t`

`char16_t` resembles `char32_t` in name, however, the characteristics of their values are very different. UTF-16 contained by `char16_t` resembles UTF-8 rather than UTF-32 contained by `char32_t`, in that UTF-16 and UTF-8 are variable-width encoding schemes, whereas UTF-32 is not. Therefore, it would be a real option that nothing is done for the time being about `char16_t` which requires special considerations, whereas `char32_t` is added into the group of `char` and `wchar_t`.

In this option, for UTF-8 and UTF-16 strings, until good treatment gets into the standard, it is encouraged for them to be converted into UTF-32 strings then passed to `std::u32regex` and regular expression algorithms.

Either way, support for `basic_regex<char32_t>` is a precondition.

IV. Technical Specifications

1. <regex>

The following changes are proposed to support `basic_regex<char32_t>`:

■ 28.1 General [re.general]

- 2 The following subclauses describe a basic regular expression class template and its traits that can handle char-like template arguments, ~~two~~^{three} specializations of this class template that handle sequences of `char` and `wchar_t`, and `char32_t` a class template ...

■ 28.3 Requirements [re.req]

- 5 [Note: ... when it is specialized for `char`, ~~or `wchar_t` or `char32_t`~~. This class template is described ...

■ 28.4 Header <regex> synopsis [re.syn]

```
typedef basic_regex<char> regex;
typedef basic_regex<wchar_t> wregex;
typedef basic_regex<char32_t> u32regex;

typedef sub_match<const char*> csub_match;
typedef sub_match<const wchar_t*> wsub_match;
typedef sub_match<const char32_t*> u32csub_match;
typedef sub_match<string::const_iterator> ssub_match;
typedef sub_match<wstring::const_iterator> wssub_match;
typedef sub_match<u32string::const_iterator> u32ssub_match;

typedef match_results<const char*> cmatch;
typedef match_results<const wchar_t*> wcmatch;
typedef match_results<const char32_t*> u32cmatch;
typedef match_results<string::const_iterator> smatch;
typedef match_results<wstring::const_iterator> wsmatch;
typedef match_results<u32string::const_iterator> u32smatch;

typedef regex_iterator<const char*> cregex_iterator;
typedef regex_iterator<const wchar_t*> wcregex_iterator;
typedef regex_iterator<const char32_t*> u32cregex_iterator;
typedef regex_iterator<string::const_iterator> sregex_iterator;
typedef regex_iterator<wstring::const_iterator> wsregex_iterator;
typedef regex_iterator<u32string::const_iterator> u32sregex_iterator;

typedef regex_token_iterator<const char*> cregex_token_iterator;
typedef regex_token_iterator<const wchar_t*> wcregex_token_iterator;
typedef regex_token_iterator<const char32_t*> u32cregex_token_iterator;
typedef regex_token_iterator<string::const_iterator> sregex_token_iterator;
typedef regex_token_iterator<wstring::const_iterator> wsregex_token_iterator;
typedef regex_token_iterator<u32string::const_iterator> u32sregex_token_iterator;
```

■ 28.7 Class template `regex_traits` [re.traits]

- 1 The specializations `regex_traits<char>`, ~~and `regex_traits<wchar_t>`~~ and `regex_traits<char32_t>` shall be valid and shall satisfy the requirements for a regular expression traits class (28.3).

`regex_traits<char32_t>`, at least the `char32_t` character names in Table 140 shall be recognized.

Table 140 — Character class names and corresponding `ctype` masks

Narrow character name	Wide character name	<code>char32_t</code> character name	Corresponding <code>ctype_base::mask</code> value
"alnum"	L"alnum"	<u>U"alnum"</u>	<code>ctype_base::alnum</code>
"alpha"	L"alpha"	<u>U"alpha"</u>	<code>ctype_base::alpha</code>
"blank"	L"blank"	<u>U"blank"</u>	<code>ctype_base::blank</code>
"cntrl"	L"cntrl"	<u>U"cntrl"</u>	<code>ctype_base::cntrl</code>
"digit"	L"digit"	<u>U"digit"</u>	<code>ctype_base::digit</code>
"d"	L"d"	<u>U"d"</u>	<code>ctype_base::digit</code>
"graph"	L"graph"	<u>U"graph"</u>	<code>ctype_base::graph</code>
"lower"	L"lower"	<u>U"lower"</u>	<code>ctype_base::lower</code>
"print"	L"print"	<u>U"print"</u>	<code>ctype_base::print</code>
"punct"	L"punct"	<u>U"punct"</u>	<code>ctype_base::punct</code>
"space"	L"space"	<u>U"space"</u>	<code>ctype_base::space</code>
"s"	L"s"	<u>U"s"</u>	<code>ctype_base::space</code>
"upper"	L"upper"	<u>U"upper"</u>	<code>ctype_base::upper</code>
"w"	L"w"	<u>U"w"</u>	<code>ctype_base::alnum</code>
"xdigit"	L"xdigit"	<u>U"xdigit"</u>	<code>ctype_base::xdigit</code>

2. <locale>

Relationship with <regex>:

- `ctype::tolower()` is called by `regex_traits::translate_nocase()`,
- `ctype::is()` is called by `regex_traits::isctype()`,
- `collate::transform()` is called by `regex_traits::transform()`,
- `collate_byname::transform()` is called by `regex_traits::transform_primary()`,
- `collate<charT>` and `collate_byname<charT>` are referred in `regex_traits::transform_primary()`. (Cf. [Library Issue 2338](#))

Thus, the following changes are proposed for support of `regex_traits<char32_t>`:

■ 22.3.1.1.1 Type `locale::category` [`locale.category`]

Table 80 — Locale category facets

Category	Includes facets
collate	<code>collate<char></code> , <code>collate<wchar_t></code> , <u><code>collate<char32_t></code></u>
ctype	<code>ctype<char></code> , <code>ctype<wchar_t></code> , <u><code>ctype<char32_t></code></u> ...

Table 81 — Required specializations

Category	Includes facets
collate	<code>collate_byname<char></code> , <code>collate_byname<wchar_t></code> , <u><code>collate_byname<char32_t></code></u>

■ 22.4.1.1.2 ctype virtual functions [locale.ctype.virtuals]

`do_toupper()` is not called by `regex_traits<char32_t>`, but the change is proposed for consistency with `do_tolower()`.

```
charT do_toupper(charT c) const;
const charT* do_toupper(charT* low, const charT* high) const;
```

- 7 *Effects*: Converts a character or characters to upper case. The second form replaces each character `*p` in the range `[low,high)` for which a corresponding upper-case character exists, with that character.

When `charT` is `char32_t`, a character or characters should be converted to upper case in conformity with the data in [UnicodeData.txt](#) provided by the Unicode Consortium.

```
charT do_tolower(charT c) const;
const charT* do_tolower(charT* low, const charT* high) const;
```

- 9 *Effects*: Converts a character or characters to lower case. The second form replaces each character `*p` in the range `[low,high)` and for which a corresponding lower-case character exists, with that character.

When `charT` is `char32_t`, a character or characters should be converted to lower case in conformity with the data in [UnicodeData.txt](#) provided by the Unicode Consortium.

```
bool do_is(mask m, charT c) const;
const charT* do_is(const charT* low, const charT* high, mask* vec) const;
```

- 1 *Effects*: Classifies a character or sequence of characters. For each argument character, identifies a value `M` of type `ctype_base::mask`. The second form identifies a value `M` of type `ctype_base::mask` for each `*p` where `(low<=p && p<high)`, and places it into `vec[p-low]`.

When `charT` is `char32_t`, the character classification should be in conformity with [Unicode Technical Standard #18, Unicode Regular Expressions, Annex C: Compatibility Properties](#).

■ 22.4.4.1 Class template `collate` [locale.collate]

- 1 ... The specializations required in Table 80 (22.3.1.1.1), namely `collate<char>`, ~~and~~ `collate<wchar_t>` and `collate<char32_t>`, apply lexicographic ordering (25.4.8).

■ 22.4.4.1.2 collate virtual functions [locale.collate.virtuals]

```
int do_compare(const charT* low1, const charT* high1, const charT* low2, const charT* high2) const;
```

- 1 *Returns*: ... The specializations required in Table 80 (22.3.1.1.1), namely `collate<char>`, ~~and~~ `collate<wchar_t>` and `collate<char32_t>`, implement a lexicographical comparison (25.4.8).

Strict Option

For `translate_nocase(charT c)` in class `regex_traits`, the C++ specification says:

- *Returns*: `use_facet<ctype<charT>> (getloc()).tolower(c)`.

However, in terms of the Unicode standard, this way is not appropriate for making a character caseless (i.e., case-folding). [Case Folding Stability](#) of Unicode says that "Case folding is not the same as lowercasing, and a case-folded string is not necessarily lowercase. In particular, as of Unicode 8.0, ..., Cherokee text case folds to the existing uppercase letters."

If we follow strictly the Unicode standard, the specification in "28.7 Class template `regex_traits` [re.traits]" is modified as follows:

```
charT regex_traits<char32_t>::translate_nocase(charT c);
```

5 *Returns:* use `facet<ctype<charT>>::tolower(c)`, if `charT` is not `char32_t`.

When `charT` is `char32_t`, if [CaseFolding.txt](#) of the Unicode Character Database provides a simple (S) or common (C) case folding mapping for `c`, then returns the result of applying that mapping to `c`; otherwise returns `c`. When the current locale is such that `tolower(U' I')` should return an integer corresponding to `U' i'` instead of `U' I'`, the mappings with status T in [CaseFolding.txt](#) may be given priority.

In this case, `regex_traits<char32_t>::translate_nocase()` does not depend upon `ctype<char32_t>::tolower()`. The proposed changes to `do_toupper()` and `do_tolower()` can be removed from this proposal document.

V. Relevant Issues

- The version of *ISO/IEC 10646* in Normative references in the C++ specification is too old. It should be replaced with a more recent version, preferably *ISO/IEC 10646:2011* or newer in which it is mentioned that UCS-2 is deprecated.
- The version of *ECMAScript Language Specification* in Normative references in the C++ specification is old. I would like to suggest replacing it with version 6.0/2015. Apparently, this specification is put in Normative references only for `<regex>`.
- ECMAScript has adopted the new regular expression `\u{h...}` where `h...` is one to six hexadecimal digits that represent a Unicode code point since version 6.0/2015. It is preferable that `<regex>` which can deal with `char32_t` (and `char16_t`) accepts this expression when the `regex_constants::ECMAScript` option is specified. This is the reason why the update in the preceding clause is suggested.

`\u{h...}` is the only new regular expression added to `RegExp` of ECMAScript since the version to which now the C++ specification refers. New additions other than expressions are the `/u` flag for Unicode support, and the `/y` flag corresponding to `regex_constants::match_continuous` that `<regex>` already has, if my understanding is correct. In other words, by supporting this expression with character-by-character matching operation for Unicode sequences, `<regex>` catches up with `RegExp` of ECMAScript 6.0/2015.

VI. References

- Ecma International, [ECMA-262 6th Edition, The ECMAScript 2015 Language Specification](#)
- The IEEE and The Open Group, [IEEE Std 1003.1, 2013 Edition](#)
- ISO/IEC [Information technology — Universal Coded Character Set \(UCS\) ISO/IEC 10646:2014](#)
- The Unicode Consortium, [FAQ - UTF-8, UTF-16, UTF-32 & BOM](#)
- The Unicode Consortium, [Unicode 8.0.0](#)
- The Unicode Consortium, *Unicode Character Database*, [CaseFolding.txt](#) (CaseFolding-8.0.0.txt)
- The Unicode Consortium, *Unicode Character Database*, [UnicodeData.txt](#)
- The Unicode Consortium, [Unicode Character Encoding Stability Policies](#)
- The Unicode Consortium, [Unicode Technical Standard #18, Unicode Regular Expressions](#)